# Finance Tracker Runbook: n8n + Postgres + Telegram

Operating notes for starting, stopping, cleaning, testing, and maintaining the project. Updated 11 November 2026.

## 1. Overview

This project is a finance bot on Telegram. Messages reach n8n through a webhook, a workflow processes them (including an AI Intent Router via OpenAI), and the results are stored in Postgres.

| Component | Role | Data stored in |
|---|---|---|
| Telegram (bot) | Interface: you send messages and receive replies | Telegram servers |
| Cloudflare Tunnel (cloudflared) | Bridge that lets Telegram reach n8n running on your laptop | Stores no data |
| n8n | Runs the workflows. Editor at http://localhost:5678 | Postgres and the n8n_data volume |
| Postgres | Finance data (accounts, budgets, savings, transactions), the activity_log, the SQL functions, and n8n's internal tables (workflows, credentials, execution history) | The postgres_data volume |

Key files in the project folder: `docker-compose.yml`, `start.sh`, `.env`, `init.sql` (tables), `functions.sql` (SQL functions, applied by `start.sh` on every start), `sanitize-workflow.sh`, and the workflow exports `finance-tracker.json` (safe for GitHub) and `finance-tracker.local.json` (contains your real User ID, never commit it).

**Important:** n8n keeps its own data (workflows, credentials, execution history) in the same Postgres database as the finance tables. Any command that wipes the whole database also wipes your workflows.

## 2. Five concepts to understand

1. **Containers are disposable; volumes hold the data.** You can stop, delete, or recreate containers at any time. Your data is safe as long as the `postgres_data` and `n8n_data` volumes are not deleted.
2. **The tunnel URL changes every time the tunnel starts.** n8n must know the latest URL (`WEBHOOK_URL`) so that Telegram calls the right address. That is why you should always start the project with `start.sh`, not with a plain `docker compose up -d` or `docker compose start`.
3. **N8N_ENCRYPTION_KEY must never be lost or changed.** It encrypts the credentials stored in n8n. Keep a copy in a password manager. If it changes, your workflows stay but the credentials can no longer be read.
4. **Save and Publish are different.** In n8n 2.x, Save only stores a draft. Publish makes that version the one Telegram messages run on. If you edit a node and do not Publish, real messages still use

the old version. Clicking Execute in the editor uses what you see in the editor, so a test can pass there and still behave differently from Telegram.

5. **A new tunnel URL needs Unpublish, then Publish.** `start.sh` gives n8n the new tunnel URL, but Telegram only learns the address when the workflow registers its webhook, which happens when it is published. After every `bash start.sh`, unpublish the workflow and publish it again (section 4).

## 3. Resuming the project (after the laptop was shut down)

1. Start Docker Desktop and wait until it reports that it is running.
2. Open the project folder in File Explorer, right-click an empty area, and choose Open Git Bash here.
3. Run:

```
bash start.sh
```

Wait until you see `Done.` and a URL like `https://....trycloudflare.com`. The script also applies `functions.sql` (safe to rerun, it never touches your data) and stops with an error message if that fails.

4. Check that all containers are up:

```
docker compose ps
```

You should see three containers: budget-postgres (healthy), budget-n8n, and budget-cloudflared.

5. Open http://localhost:5678 and log in. If the editor was already open in your browser before `start.sh`, refresh the page.
6. Unpublish, then Publish the finance-tracker workflow (section 4). This is the step that registers the new tunnel URL with Telegram. Make sure it shows Published again.
7. Test: send `/start` or one message to the bot on Telegram, then check the Executions tab in n8n.
8. If the bot stays silent: check the webhook with `getWebhookInfo` (section 4), Unpublish and Publish again, then check the logs with `docker compose logs n8n`.

> *After a laptop restart, the postgres and n8n containers may start by themselves (because of restart: unless-stopped) with an outdated tunnel URL. Still run bash start.sh: it starts a fresh tunnel and recreates n8n with the correct URL.*

## 4. After every start.sh: Unpublish, then Publish (tunnel webhook)

The free Cloudflare quick tunnel gets a new random URL (`https://xxxx.trycloudflare.com`) every time it starts. `start.sh` passes that URL to n8n as `WEBHOOK_URL`, but Telegram does not read it from there: n8n tells Telegram the address (through `setWebhook`) at the moment the workflow is published. A workflow that stayed published through a restart can keep Telegram pointing at the old, dead URL, so the bot looks silent even though everything is green. Unpublishing and publishing again forces n8n to register the new URL. In this setup, treat that step as mandatory every time a new tunnel starts.

The steps:

1. Run `bash start.sh` and wait for `Done.` and the new tunnel URL.
2. Open http://localhost:5678 (refresh if it was already open) and open the finance-tracker workflow.

3. Click Unpublish and confirm. The workflow goes back to a draft and the bot stops replying; that is expected.
4. Click Publish. If an error such as `Bad Request: bad webhook ... Failed to resolve host` appears, the new tunnel address is not reachable yet: wait 20 to 30 seconds and Publish again.
5. Check the registered webhook (below), then send one test message such as `/start`.

Check the registered webhook. Replace `<TOKEN>` with your bot token. Never paste the real token into chats, screenshots, or commits.

```
curl "https://api.telegram.org/bot<TOKEN>/getWebhookInfo"
```

Read three fields in the JSON answer: `url` must start with the tunnel URL printed by `start.sh`, `pending_update_count` should be 0 or drop quickly, and `last_error_message` should be absent.

| What getWebhookInfo shows | Meaning and fix |
|---|---|
| `url` shows an old trycloudflare address | The webhook was not re-registered. Unpublish, Publish again, then run getWebhookInfo once more. |
| `url` is empty | No webhook is registered: the workflow is not published. Publish it. |
| `last_error_message`: Wrong response from the webhook: 530 (or a Cloudflare 1033 page) | Telegram called a tunnel that is no longer connected (the old URL). Same fix: Unpublish, then Publish. |
| `last_error_message`: 404 | n8n is reachable, but no published workflow owns that path (the workflow is unpublished, or the Telegram Trigger was changed). Publish, or check the Telegram Trigger node. |
| `last_error_message` about a timeout | The tunnel is slow or down. Check `docker compose logs cloudflared`, then run `bash start.sh` again. |
| `pending_update_count` keeps growing | Messages are waiting because Telegram cannot deliver them. Fix the cause above; the old messages arrive (possibly late) once the webhook works. |

Do not start or restart n8n with a plain `docker compose up` or `docker compose restart n8n`. `WEBHOOK_URL` then falls back to `http://localhost:5678`, which Telegram cannot reach (it needs public HTTPS). Always go through `bash start.sh`.

> *Rule of thumb: new tunnel means new URL means Unpublish, then Publish. Editing a node means Save, then Publish (no Unpublish needed). Shutting down with docker compose stop needs no Unpublish.*

## 5. Shutting down the project (end of a work session)

1. If you changed the workflow today: export it from n8n (three-dot menu, Download), save it in the repo, and commit.
2. In the project folder, run:

```
docker compose stop
```

This stops all containers without deleting anything. Your data, workflows, and credentials stay safe.

3. Verify with `docker compose ps -a`. Every container should show Exited.
4. It is now safe to close Docker Desktop or shut down the laptop.

Note: shutting down the laptop without running `docker compose stop` usually does not damage data (Postgres has crash recovery), but stopping first is safer. While the project is off, the bot will not reply; that is expected.

## 6. stop, down, and down -v: what is the difference

| Command | Containers | Volumes (data) | Use it for |
|---|---|---|---|
| `docker compose stop` | Stopped, still exist | Safe | Ending a daily work session |
| `docker compose down` | Removed (along with the network) | Safe | Cleaning up after changing docker-compose.yml, or when a container is stuck |
| `docker compose down -v` | Removed | **DELETED** | A full reset, starting from zero |

When to use `docker compose down` (data stays safe; start again with `bash start.sh`):

- After changing the structure of docker-compose.yml, for example renaming a service, changing a port, or removing a service.
- When you get a `container name already in use` error or a container is stuck.
- When you want to clean up containers and the network without touching data.

When to use `docker compose down -v` (all data is lost, so back up first):

- Changing `POSTGRES_USER`, `POSTGRES_PASSWORD`, or `POSTGRES_DB` when the volume already exists. Postgres only reads these values when the volume is first created, so editing `.env` has no effect until the volume is deleted.
- Changing `init.sql` and wanting the tables recreated from scratch (`init.sql` only runs when the volume is empty). For a small addition such as a new table, section 7 is enough.
- Testing a clean installation, for example to confirm this documentation can be followed from the beginning.
- The data is corrupted and starting over is easier.

Before `down -v`: back up the database (section 9) and export the workflow. After `down -v`: run `bash start.sh`, then import the workflow, recreate the credentials in n8n, and Publish (section 4).

Remember: `down -v` also deletes your n8n workflows and credentials, because n8n keeps them in the same Postgres database. If you only want to clear test data, use section 7.

## 7. Clearing test data without a full reset

Before running a test plan, you usually only need empty finance tables, not a brand new Docker environment. A full reset (`down -v`) also deletes the workflow, the credentials, and the webhook registration, so you would have to import, reconnect credentials, and Publish all over again.

| Situation | What to use |
|---|---|
| Test data from a previous round is still there; schema and workflow are fine | TRUNCATE the finance tables (below). The workflow, credentials, and indexes stay intact. |
| A container is stuck or its name is in use | `docker compose down`, then `bash start.sh`. Data stays safe. |
| You changed `init.sql` (a column, constraint, or index) or added a table | Apply the change by hand with ALTER TABLE or CREATE, or run `init.sql` again (below, it is safe to rerun). `down -v` only if you are fine losing everything, because `init.sql` only runs on an empty volume. |
| You changed `POSTGRES_USER`, `POSTGRES_PASSWORD`, or `POSTGRES_DB` | `down -v` (back up first). Postgres only reads those values when the volume is first created. |
| You want to test a clean installation from zero | `down -v`, then follow section 3 and import the workflow. |
| Credentials cannot be decrypted | Restore the correct `N8N_ENCRYPTION_KEY` first. Reset only as a last resort. |

To empty the finance data only, open the database:

```
docker compose exec postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
```

then run:

```
TRUNCATE transactions, savings, budgets, accounts, activity_log RESTART IDENTITY CASCADE;
```

- **What it does:** deletes every row in the five bot tables (accounts, budgets, savings, transactions, activity_log) and resets the ID counters to 1, so the IDs in the test plan checkpoints match. Indexes and constraints stay.
- **What stays safe:** the workflow, credentials, execution history, and the tunnel and Published status.
- **What you must not touch:** n8n's own tables (for example `workflow_entity`, `credentials_entity`, `execution_entity`). Never TRUNCATE or DROP those, and never run `DROP SCHEMA public`.

Verify: every number must be 0.

```
SELECT (SELECT count(*) FROM accounts) AS accounts, (SELECT count(*) FROM budgets) AS budgets, (SELECT count(*) FROM savings) AS savings, (SELECT count(*) FROM transactions) AS transactions, (SELECT count(*) FROM activity_log) AS activity_log;
```

Check that the schema still matches `init.sql` (only needed when something looks off):

```
\dt
SELECT indexname FROM pg_indexes WHERE tablename IN ('accounts','budgets','savings','transactions','activity_log') ORDER BY 1;
```

You should see the five bot tables among n8n's tables, and indexes including `uq_accounts_name`, `uq_budgets_category`, `uq_savings_name`, `uq_transactions_source_key`, and the two

`idx_activity_log_*` indexes. If something is missing, apply it as described below. Re-creating the database is the very last option. Back up first (section 9) if the data matters.

**Adding a table or index to an existing database** (for example `activity_log` when your database was created before it existed): `init.sql` only runs on an empty volume, but every statement in it uses `IF NOT EXISTS`, so you can run the whole file again without losing data. Run this from Git Bash in the project folder:

```
docker compose exec -T postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -v ON_ERROR_STOP=1' < init.sql
```

Checklist before starting a test plan:

1. `bash start.sh`, then wait for the new tunnel URL.
2. Unpublish, then Publish the workflow, and check `getWebhookInfo` (section 4).
3. Empty the bot tables with TRUNCATE and make sure every count is 0.
4. Send `halo` to the bot (smoke test) and make sure it replies with the fixed greeting. Note that this creates the first row in `activity_log`.
5. Start the test plan from the first row.

## 8. Everyday commands

| Goal | Command |
|---|---|
| Show container status | `docker compose ps -a` |
| Show n8n logs (Ctrl+C to exit) | `docker compose logs -f n8n` |
| Show tunnel logs | `docker compose logs cloudflared` |
| Validate docker-compose.yml | `docker compose config` |
| Open the database | `docker compose exec postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'` |
| Apply `functions.sql` after editing it | `bash start.sh` (it runs the file on every start) |
| Stop the tunnel only (when not demoing) | `docker compose stop cloudflared` |
| Update n8n to the latest version | `docker compose pull n8n`, then `bash start.sh` (back up first). The image is not pinned, so consider pinning a version tag in `docker-compose.yml`. The n8n 3.0 Migration report is informational: read it, then decide when to upgrade |
| Check the Telegram webhook (replace <TOKEN>) | `curl "https://api.telegram.org/bot<TOKEN>/getWebhookInfo"` |
| Restart n8n properly | `bash start.sh` (not `docker compose restart n8n`), then Unpublish and Publish |
| Empty the bot data only (inside psql) | `TRUNCATE transactions, savings, budgets, accounts, activity_log RESTART IDENTITY CASCADE;` |

| Goal | Command |
|---|---|
| Row counts of the five bot tables (inside psql) | `SELECT (SELECT count(*) FROM accounts) AS accounts, (SELECT count(*) FROM budgets) AS budgets, (SELECT count(*) FROM savings) AS savings, (SELECT count(*) FROM transactions) AS transactions, (SELECT count(*) FROM activity_log) AS activity_log;` |
| Show the latest bot activity (inside psql) | `SELECT log_id, created_at, status, intents, left(message, 60) AS message FROM activity_log ORDER BY log_id DESC LIMIT 20;` |

Inside psql: `\dt` lists the tables, `SELECT * FROM accounts;` shows a table, and `\q` quits.

## 9. Backup, restore, and sharing the workflow

Full backup of the database (safe from both PowerShell and Git Bash):

```
mkdir backups
docker compose exec -T postgres sh -c 'pg_dump --clean --if-exists -U "$POSTGRES_USER" -d "$POSTGRES_DB" -f /tmp/backup.sql'
docker compose cp postgres:/tmp/backup.sql ./backups/backup.sql
```

This deliberately avoids > in PowerShell, because older PowerShell versions save redirected output as UTF-16, which breaks the SQL file when you restore it.

Finance data only (accounts, budgets, savings, transactions as INSERT statements, handy for keeping or moving your data without n8n's tables):

```
docker compose exec -T postgres sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" --data-only --column-inserts -t accounts -t budgets -t savings -t transactions -f /tmp/finance-data.sql'
docker compose cp postgres:/tmp/finance-data.sql ./backups/finance-data.sql
```

Restore the full backup (stop n8n first so it does not write during the restore):

```
docker compose stop n8n cloudflared
docker compose cp ./backups/backup.sql postgres:/tmp/backup.sql
docker compose exec -T postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -f /tmp/backup.sql'
bash start.sh
```

The full backup contains the whole database: your finance data, the activity log (message texts and replies), plus the n8n workflows and credentials (credentials are encrypted with `N8N_ENCRYPTION_KEY`). Keep it private. After a restore, `bash start.sh` runs again, so also Unpublish and Publish (section 4).

Export the workflow: three-dot menu in n8n, Download. Store it in the GitHub repo (the JSON contains no secrets). Never commit `.env` or the `backups/` folder, because backup files contain your financial data.

**Sharing the workflow on GitHub.** The export contains your real Telegram User ID in the Telegram Trigger node. Before committing:

1. Save the export as `finance-tracker.local.json` (this name is ignored by git through `.gitignore`).
2. Run:

```
bash sanitize-workflow.sh finance-tracker.local.json
```

3. The script writes `finance-tracker.json` with the User ID replaced by `YOUR_TELEGRAM_USER_ID` and the `instanceId` blanked, then aborts if a User ID, bot token, `sk-` API key, or `trycloudflare.com` URL remains. Commit only this file, plus `.env.example` (without real values).
4. `.gitignore` already ignores `.env`, `*.local.json`, the `backups/` folder, and `*.sql.gz`. Do not force-add them.

## 10. What the bot understands (commands and keywords)

The bot has two commands and understands everything else as plain language. Both commands reply with fixed text, so they cost no AI calls and never touch the database.

| Command | What it does |
|---|---|
| `/start` | Introduction: how the bot works in three steps, and the short rules (message types, accounts first and budgets optional, amount limits, dates, combined messages, no edit or undo). |
| `/help` | The usage guide: format and keywords for Financial Setup, recording transactions, and Financial Insights. The words `bantuan`, `panduan`, and `cara pakai` do the same. |

In BotFather, send `/setcommands` and paste this list so Telegram shows the commands in its menu:

```
start - How the bot works and its rules
help - Format and keywords for every feature
```

How messages are sorted (the intent router reads the keywords):

| Type | Trigger words | Examples |
|---|---|---|
| Record a transaction | makan, beli, bayar, gaji masuk, dapat, terima, bonus, untung, transfer, kirim, pindah, top up, tabung, setor | `makan siang 50rb pakai BCA`, `transfer 500rb dari BCA ke OVO` |
| Financial setup | buat, bikin, ubah, ganti, set, tambah, kurangi, hapus + account, budget, or savings | `buat akun BCA tipe bank saldo awal 5jt`, `ubah deadline savings rumah jadi juni 2028` |
| Insight | a question about balance, budgets, savings, or transactions, or a what-if (`kalau aku beli ...`) | `sisa uangku berapa?`, `kalau aku beli laptop 10jt gimana?` |

- Accounts need a name, a type (`cash`, `bank`, `e-wallet`, `credit card`, `loan`, `other`), and a starting balance. Budgets need a name and a monthly limit. Savings need a name and a target; the deadline is optional.
- What can be changed later: accounts (name, type, balance), budgets (name, limit), savings (name, target, deadline), and each can be deleted. Bulk deletes ("hapus semua akun") are blocked.
- Amount units: `rb` / `ribu` / `k`, `jt` / `juta`, `M` / `miliar`. One transaction must be between Rp1 and Rp100 billion.
- **Budgets are optional.** An expense without a matching budget is still recorded (category from the AI, or `other`) and the reply says no budget was set. Only an expense that names a budget that does not exist is rejected.
- **Setup needs a create word.** Creating an account, budget, or savings goal needs a word such as `buat`, `bikin`, `set`, `atur`, or `tambah akun`. Without it the bot answers with a fallback message and creates nothing (for example `BCA tipe bank saldo awal 10jt`). `tambahin 10k di akun BCA` adjusts a balance, while `tambahin akun baru ...` creates one. Advice questions such as `enaknya nama budget apa ya?` are answered as insights and never trigger setup.
- **Income category.** The AI reads the source from the message (`gaji`, `bonus`, `freelance`, `komisi`, `dividen`, `cashback`, `jual ...`). If it leaves the category empty, a small word list in the Normalize Transactions node fills it from the message. If no source word is found, the income is recorded as `other` and the reply asks you to name the source.
- **Naming a budget on purpose.** A budget is chosen explicitly only with the words `kategori <name>` or `masuk ke budget <name>`. If that budget does not exist the expense is rejected. Writing only `budget coffee` is not enough, so `beli kopi 20rb pakai BCA budget coffee` is recorded under food through the word `kopi`.
- **Tables and how-to answers.** Requests for details (`rincian`) are answered with a table printed by the app from database rows, in a monospace code block, and can be filtered by category (`pengeluaran shopping`). Questions about how to use the bot (`cara hapus akun`) are answered from a built-in guide and end with a pointer to `/help`. `halo`, `hi`, and `kamu bisa apa?` get fixed replies.
- A message may hold several things at once (up to 10 transactions, plus setup commands and questions). The replies arrive as separate messages per type.

## 11. Dates, months, and time zone

- The containers run in the Asia/Jakarta time zone (`TZ`, `PGTZ`, `GENERIC_TIMEZONE`), so "today" and the change of month follow WIB: the month changes at 00:00 WIB.
- **Every transaction gets a `transaction_date`.** It is the date written in the message, a relative date resolved by code from today's date (`kemarin`, `2 hari lalu`, `2 days ago`, `seminggu lalu`), or today when no date is given.
- Budget usage, over-limit warnings, monthly income and savings totals, and the insights all follow `transaction_date`. A transaction dated 20 October counts toward October even if you record it in November. Only old rows without a date fall back to `created_at`.
- **At most 60 days back.** Older dates, vague phrases such as `bulan lalu` or `2 bulan lalu`, and future dates (`besok`, `lusa`, `3 hari lagi`, `tomorrow`, `in 3 days`) are rejected with a message and nothing is recorded (balances do not change). Without this check a message such as `makan`

`siang 50rb pakai BCA besok` would have been recorded with today's date. In a message with several transactions, only the affected transaction is rejected.

- Not understood: weekday names (`senin lalu`), `tadi malam`, and dates without a year. A message with two different relative dates is recorded with today's date.
- Setup dates ("tahun depan") are resolved by the parser from today's date. For savings deadlines, a year alone means 31 December, and a date that does not exist (31 Februari) is answered with an invalid-date message.
- Insight answers are written by the AI from database figures, so check month-based answers with SQL when it matters.
- Old rows recorded before this behaviour may have an empty `transaction_date`. This fills them from `created_at` (safe to run once):

```
UPDATE transactions SET transaction_date = created_at WHERE transaction_date
IS NULL;
```

## 12. Fixing wrongly recorded data

The bot has no edit or undo for transactions. Fix mistakes directly in Postgres, and always reverse the balance effect together with the row, in one database transaction (`BEGIN` … `COMMIT`). Setup mistakes (account, budget, or savings values) can simply be corrected with the bot, for example `set saldo BCA 2jt` or `ubah budget food jadi 700rb`.

First find the transaction ID (open psql as in section 8):

```
SELECT transaction_id, transaction_date, transaction_type, amount, category,
notes, from_account_id, to_account_id, savings_id FROM transactions ORDER BY
transaction_id DESC LIMIT 10;
```

Then reverse it according to its type. Replace `42` with the real `transaction_id`. After that, send the corrected message to the bot again if you still want it recorded.

| Type | What to reverse |
|---|---|
| expenses | Give the amount back to `from_account_id` |
| income | Take the amount out of `to_account_id` |
| transfer | Give it back to `from_account_id` and take it out of `to_account_id` |
| savings | Give it back to `from_account_id` and reduce `current_amount` of the savings goal |

Expense:

```
BEGIN;
UPDATE accounts a SET balance = a.balance + t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.from_account_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

Income:

```
BEGIN;
UPDATE accounts a SET balance = a.balance - t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.to_account_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

Transfer:

```
BEGIN;
UPDATE accounts a SET balance = a.balance + t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.from_account_id;
UPDATE accounts a SET balance = a.balance - t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.to_account_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

Savings deposit:

```
BEGIN;
UPDATE accounts a SET balance = a.balance + t.amount, updated_at = NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND a.account_id =
t.from_account_id;
UPDATE savings s SET current_amount = s.current_amount - t.amount, updated_at
= NOW()
  FROM transactions t WHERE t.transaction_id = 42 AND s.savings_id =
t.savings_id;
DELETE FROM transactions WHERE transaction_id = 42;
COMMIT;
```

If only the date is wrong (budget usage follows the date, so this moves the transaction to the right month):

```
UPDATE transactions SET transaction_date = '2026-11-05' WHERE transaction_id
= 42;
```

Afterwards check the balances with `SELECT account_name, balance FROM accounts;` and ask the bot `sisa uangku berapa?`. The two should agree.

## 13. Reading the activity log

Every reply the bot sends is also written to `activity_log`: the Telegram update ID, chat ID, your message, the detected intents, a status, and the reply text. Statuses: `succeeded`, `rejected` (something was refused, for example an unknown account), `mixed` (part accepted, part refused), and `error`. If the log insert itself fails, the bot still replies.

```
SELECT log_id, created_at, status, intents, left(message, 60) AS message FROM
activity_log ORDER BY log_id DESC LIMIT 20;
```

Count by status:

```
SELECT status, count(*) FROM activity_log GROUP BY status ORDER BY 2 DESC;
```

Find out why something did not get recorded (what you sent and what the bot answered):

```
SELECT log_id, created_at, message, reply FROM activity_log WHERE status IN
('rejected','mixed','error') ORDER BY log_id DESC LIMIT 20;
```

The log contains your message texts and amounts. It stays in your local database, so do not commit database dumps.

## 14. Test plans and what testing taught

The bot was tested in three rounds. Each test plan lists the message to send, the expected result, and a place for the actual result. The adjusted result sheets record what happened and what was changed afterwards. The test plans are guides. Their expected columns were written before the final fixes, so the result sheets are the up-to-date record.

| Test plan | Scope | Passed first run | Fixed after |
|---|---|---|---|
| 1st: Financial Setup | Accounts, budgets, savings: create, update, adjust, delete, validation, date handling | 78 of 85 | 7 |
| 2nd: Transactions and Insights | Income, expense, transfer, savings, negative cases, insights, multi-intent, extra edge cases | 71 of 80 | 9 |
| 3rd: GitHub version | Fixed commands, create-word guard, optional budgets, negative cases, tables, how-to and advice questions, combined messages | 56 of 57 | 1 (future dates), plus 1 hardening (income source) |

The 3rd round passed in all places except one: `besok` was recorded with today's date, and it is fixed in Normalize Transactions (section 10). The income-source word list was added after `freelance 1,5jt ke 0V0` was recorded as `other`. Two minor notes remain: an advice answer may repeat a budget name, and the status-budget wording can vary. Neither changes any number.

Things worth remembering when you run a test plan:

- Run the phases in order, each on empty tables (section 7). Transactions need the accounts, budgets, and savings from the setup phase.
- Use English budget names (`food`, `transport`, `entertainment`) so the parser's category names match your budgets. A category that matches none of your budgets is rejected with a suggestion.
- Check three places for every message: the Telegram reply, the Executions tab in n8n, and the database.
- An expense needs a named source account, and every transaction in a multi-transaction message is validated on its own.
- A what-if question must not create any transaction. Confirm with `SELECT count(*) FROM transactions;` before and after.

- A deposit to savings without a goal name goes to the goal called `general`, so create it first.
- Unit and size checks: `1M` is one billion rupiah. An income above Rp100 billion (`uang bonus 1T masuk ke BCA`) is rejected for its size, while a huge expense (`beli rumah 5T pakai BCA`) is rejected because the balance is too low.
- Hybrid inputs (English, Indonesian, Chinese in one message) can still misfire. Record the result as a finding and, if needed, fix it in the prompt.

## 15. Troubleshooting

| Symptom | Cause and fix |
|---|---|
| `bash start.sh` error `\r` or `bad interpreter` | Windows line endings (CRLF). Run `sed -i 's/\r$//' start.sh` |
| `Cannot connect to the Docker daemon` or `docker: command not found` | Docker Desktop is not running |
| Failed to get the tunnel URL | Check `docker compose logs cloudflared` and your internet connection |
| `start.sh` says functions.sql failed to apply | Read the SQL error printed above the message, fix `functions.sql`, and run `bash start.sh` again. The file never changes your data. |
| Bot is silent, Executions tab is empty | Unpublish, then Publish the workflow (section 4) and check getWebhookInfo. Make sure you started through `start.sh` (latest URL) and that the User ID in the Telegram Trigger is correct |
| Red execution on a Postgres node | Postgres credential: the host must be `postgres`, not `localhost` |
| Red execution on the Log Activity node, or `relation "activity_log" does not exist` | The table is missing (the database was created before it was added). Run `init.sql` again as shown in section 7. The bot still replies while the log fails. |
| Red execution on an OpenAI node | Wrong API key or no credit left |
| `container name already in use` | Run `docker compose down`, then `bash start.sh` |
| Warning that a `variable is not set` | A `$` character in a `.env` value. Write `$$`, or wrap the value in single quotes |
| n8n credential cannot be decrypted | `N8N_ENCRYPTION_KEY` differs from when the credential was created. Restore the old key, or recreate the credential |
| A table does not exist | The volume existed before `init.sql` was mounted. Run `init.sql` manually (section 7), or reset with `down -v` |
| Bot is silent after a fresh `start.sh` although the workflow shows Published | The new tunnel URL is not registered with Telegram. Unpublish, Publish, then check getWebhookInfo (section 4) |
| I changed a node but the bot still behaves the old way | The change is only a draft until you Publish. Save, then Publish |
| It works in the editor (Execute) but not from Telegram | The editor runs the draft, Telegram runs the published version. Publish |

| Symptom | Cause and fix |
|---|---|
| Telegram stopped replying after `docker compose up` or restart outside `start.sh` | n8n started with `WEBHOOK_URL=http://localhost:5678`. Run `bash start.sh`, then Unpublish and Publish |
| Telegram error `can't parse entities` | Text with the characters `_ * [` reached Telegram as Markdown. The Send node escapes them for normal replies; if you edit the `/start` or `/help` text, avoid those characters or keep backticks in pairs |
| Tables show raw triple backticks in Telegram | The Send node must not escape code blocks. It splits the text on `` ``` blocks and escapes _*[` only outside them |
| Red execution on a Telegram node: `The connection to the server was closed unexpectedly` or a timeout | Usually a transient network error. Check from inside the container: `docker compose exec n8n wget -qO- --timeout=10 https://api.telegram.org`. Keep Retry On Fail on every Telegram node, and make sure `NODE_OPTIONS=--dns-result-order=ipv4first` is under `environment:` of the n8n service. After a sleep or network change, run `bash start.sh` |
| A created item is missing, or the bot says the setup message is incomplete | Creation needs a create word (section 10). Rewrite with `buat` or `bikin` |
| `start.sh` stops at functions.sql and an old function keeps running | A syntax error in `functions.sql` (a common one is `ELSE IF` instead of `ELSIF`). Fix it and run `bash start.sh` again |
| An amount was recorded wrongly (`1M` became 1jt, or `40k` and `10k` were added together) | The AI parser misread the message. Record the amounts one per line, or reverse it with SQL (section 12). A guard that cross-checks amounts against the original message is on the roadmap |
| The bot says "Aku belum paham pesan itu" for a valid message | The router found no intent. Rephrase with the keywords in `/help`, or check the AI Intent Router execution |
| An income shows category `other` although I named a source | The source word may not be in the word list. Say it in the common form (`gaji`, `bonus`, `freelance`, ...) or add a line to `INCOME_WORDS` in Normalize Transactions (section 10) |
| A transaction with `besok` was recorded with today's date | The future-date check is missing from Normalize Transactions. Check that `relativeDate()` returns `FUTURE` for `besok`, `lusa`, `tomorrow`, and that the `problem` line maps it to `future`, then Publish again |
| A transaction was rejected for its date | The date is older than 60 days, vague (`bulan lalu`), or in the future. Write a full date inside the last 60 days, or `kemarin` / `5 hari lalu` (section 11) |
| An insight shows a number that does not match the database | Figures come from the database, but the AI writes the wording. If a ⚠️ note appears, a number was not in the data. Check with SQL (section 8) and report it as a finding |

| Symptom | Cause and fix |
|---|---|
| A recorded transaction is wrong | There is no edit or undo. Reverse it in SQL (section 12) |

## 16. Golden rules

- Never run `docker compose down -v` unless you really want to delete all data, including n8n workflows and credentials.
- Always start through `bash start.sh`, then Unpublish and Publish.
- Keep `N8N_ENCRYPTION_KEY` in a password manager.
- Never commit `.env` or backups, and never paste tokens or passwords in chats or screenshots. Commit only the sanitized `finance-tracker.json`.
- Set a spending limit on your OpenAI account.
- Stop the tunnel when you are not demoing, because the n8n editor is reachable through the tunnel URL too.
- Make sure the Telegram Trigger is restricted to your User ID.
- Never edit transaction rows without reversing the balance in the same database transaction.